

RELATÓRIO TÉCNICO FINAL

Benchmark reproduzível de métodos clássicos de aprendizado de máquina para diagnóstico de falhas no Tennessee Eastman Process

Subprojeto WP1A — Grupo A

MEI0028 — Modelagem e Simulação

Programa de Pós-Graduação em Engenharia Industrial e Inteligência Artificial

Pontifícia Universidade Católica de Goiás

Anderson Ricardo de Freitas Rocha

Professor responsável: Clarimar José Coelho

Goiânia, setembro de 2026

1 OBJETIVO

Medir e comparar, sob um único protocolo experimental declarado por inteiro, o desempenho, o equilíbrio entre classes e o custo computacional de seis famílias clássicas de aprendizado de máquina no diagnóstico de falhas do Tennessee Eastman Process, usando a base pública de Rieth et al. (2017).

O experimento responde a cinco perguntas, formuladas no WP1A:

QP1. Qual das seis famílias obtém o melhor desempenho global sob divisão por execução?

QP2. Quais falhas permanecem difíceis de distinguir, e de quê?

QP3. Quanto a unidade de divisão dos dados altera o resultado medido?

QP4. Qual o custo computacional de cada família, no mesmo equipamento?

QP5. Qual família oferece o melhor compromisso entre desempenho e custo?

2 FUNDAMENTAÇÃO TEÓRICA

2.1 O processo e o problema

O Tennessee Eastman Process é um simulador de planta química proposto por Downs e Vogel (1993) a partir de um processo industrial real da Eastman Chemical Company. A planta produz dois produtos a partir de quatro reagentes gasosos, com reator, condensador, separador, compressor e coluna de esgotamento operando em malha fechada. O simulador injeta vinte perturbações programadas (IDV1 a IDV20), que vão de degraus na composição de alimentação a emperramento de válvula.

O diagnóstico de falhas consiste em, a partir das leituras dos sensores, decidir em que condição a planta se encontra. Formulado como classificação supervisionada, cada vetor de 52 variáveis medidas num instante é atribuído a uma de 21 classes: a operação normal e as vinte falhas.

2.2 Vazamento de dados por correlação temporal

Leituras consecutivas de uma mesma execução são quase idênticas, porque o processo tem inércia física. Se as amostras forem sorteadas individualmente entre treinamento e teste, o modelo encontra no teste vizinhos temporais das amostras que já viu, e o desempenho medido passa a refletir memorização, não generalização. A literatura de reprodutibilidade documenta esse mecanismo como uma das formas mais frequentes de vazamento (KAPOOR; NARAYANAN, 2023; SAEB et al., 2017; HAMMERLA; PLÖTZ, 2015).

A prevenção consiste em adotar a execução completa como unidade indivisível de divisão: todas as amostras de uma execução vão para o mesmo conjunto.

2.3 Métricas sob desbalanceamento

A acurácia é enganosa quando as classes têm suporte desigual. Este trabalho adota como principais o F1 macro, média aritmética do F1 de cada classe, que dá o mesmo peso a classes raras e frequentes, e o coeficiente de correlação de Matthews multiclasse

(GORODKIN, 2004), que considera as quatro células da matriz de confusão e só se aproxima de 1 quando o modelo acerta em todas as classes (CHICCO; JURMAN, 2020). A acurácia balanceada (BRODERSEN et al., 2010) é reportada como terceira referência.

3 MATERIAIS

3.1 Base de dados

Rieth, Amsel, Tran e Cook (2017), *Additional Tennessee Eastman Process Simulation Data for Anomaly Detection Evaluation*, Harvard Dataverse, DOI 10.7910/DVN/6C3JR1, domínio público. São quatro arquivos no formato .RData, com 1,40 GB no total, cujos MD5 publicados foram conferidos no ato do download.

Tabela 1 — Composição da base

Arquivo	Linhas	Classes	Execuções por classe	Amostras por execução
TEP_FaultFree_Training	250.000	1	500	500
TEP_FaultFree_Testing	480.000	1	500	960
TEP_Faulty_Training	5.000.000	20	500	500
TEP_Faulty_Testing	9.600.000	20	500	960

Fonte: auditoria própria (reports/01-auditoria.md).

Cada arquivo tem 55 colunas: faultNumber, simulationRun, sample e as 52 variáveis de processo, sendo 41 medidas (XMEAS) e 11 manipuladas (XMV), amostradas a cada três minutos. Nos arquivos de treinamento a falha é introduzida uma hora após o início da execução, o que corresponde à amostra 20; nos de teste, oito horas, o que corresponde à amostra 160.

3.2 Equipamento

Computador Apple com processador M3, oito núcleos (quatro de desempenho e quatro de eficiência) e 8 GB de memória, sob macOS 26.5.1. Todas as medidas de tempo, memória e tamanho de modelo foram tomadas exclusivamente nesta máquina. O ambiente Google Colab, com GPU T4, foi usado apenas para a busca de hiperparâmetros do XGBoost, e os tempos ali obtidos não entram em nenhuma comparação.

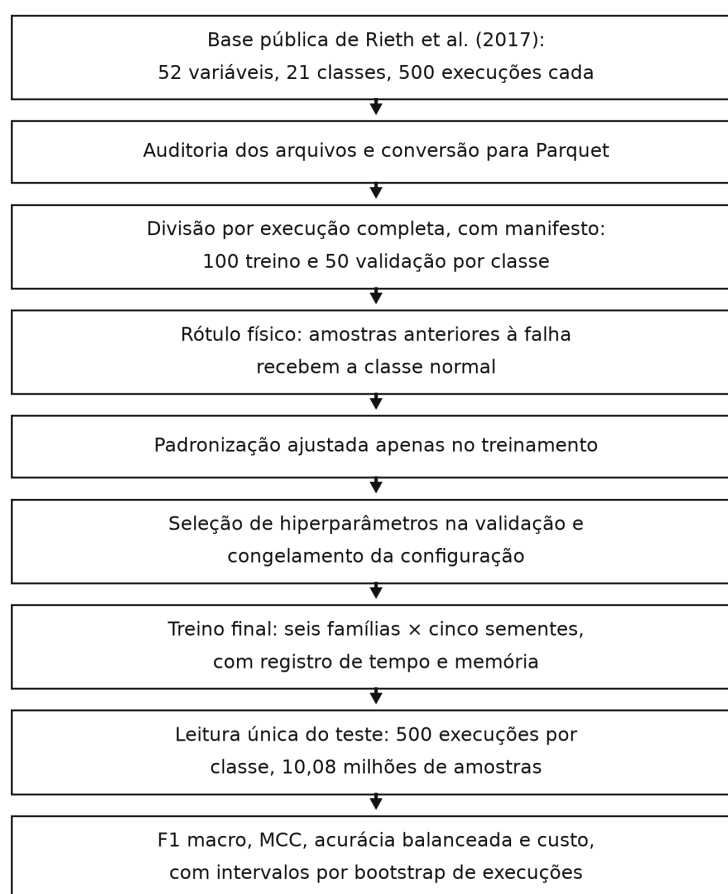
3.3 Software

Python 3.11.15, com numpy 2.4.6, pandas 3.0.5, scikit-learn 1.9.0, xgboost 3.2.0 e scipy 1.17.1. A conversão dos arquivos .RData foi feita em R, por classe, com escrita incremental em Parquet. O registro completo de versões está em results/metadata/ambiente.json e em requirements.txt.

4 PROCEDIMENTO EXPERIMENTAL

O procedimento foi executado na ordem abaixo, um script por etapa, todos no repositório público. A Figura 1 resume o encadeamento.

Figura 1 — Etapas do protocolo experimental



Fonte: Autores (2026).

Etapa 1 — Obtenção e conferência da base. Download dos quatro arquivos a partir do Harvard Dataverse e verificação dos MD5 publicados (`01_download.py`). Conversão para Parquet em ponto flutuante de 32 bits, feita em R por classe, com escrita incremental, para manter o uso de memória constante (`convert_rdata.py`).

Etapa 2 — Auditoria. Verificação de dimensões, nomes e tipos das colunas, distribuição das classes, número de execuções por classe, amplitude de `sample`, valores ausentes, infinitos e duplicatas de chave (`02_auditoria.py`).

Etapa 3 — Divisão dos dados. Sorteio, com semente 42, de 100 execuções por classe para treinamento e 50 para validação, a partir do arquivo de treinamento. O arquivo de teste foi reservado por inteiro, com suas 500 execuções por classe. A unidade sorteada é a execução completa. O manifesto resultante foi gravado e a interseção entre os três conjuntos verificada como vazia (`03_split.py`).

Etapa 4 — Rotulagem. Como a falha só entra depois do início de cada execução, as amostras anteriores correspondem fisicamente à operação normal. Adotou-se esse estado físico como rótulo primário: amostras com `sample` até 20 no treinamento, e até 160 no teste, recebem a classe 0. As métricas com o rótulo da execução inteira também foram calculadas, para permitir comparação com trabalhos que adotam essa convenção.

Etapa 5 — Análise exploratória. Estatísticas descritivas, correlação entre variáveis, análise de componentes principais e distância média de cada falha ao regime normal (05_edu.py).

Etapa 6 — Experimento de controle do vazamento. Com 25 execuções por classe, árvore de decisão e Random Forest foram treinados duas vezes, com volume de dados idêntico: uma vez sob divisão por execução e outra sob divisão aleatória por amostra. A diferença entre as duas condições mede o efeito do vazamento nesta base. O experimento foi repetido em dez seleções independentes de execuções (04_piloto_vazamento.py, 04b_piloto_vazamento_repeticoes.py).

Etapa 7 — Seleção de hiperparâmetros. Grade fatorial reduzida, de 4 a 12 configurações por modelo, avaliada pelo F1 macro sobre as 50 execuções por classe da validação. O conjunto de teste não foi consultado em nenhum momento desta etapa. A configuração vencedora de cada modelo foi gravada e congelada (06_busca_hp.py, 06b_merge_hp.py, 06c_busca_hp_lr.py).

Etapa 8 — Treinamento final. Cada um dos seis modelos foi treinado cinco vezes, com as sementes 42, 123, 2024, 7 e 555, mantendo fixas as execuções de treinamento. Cada par modelo-semente rodou em processo próprio, para que a medida de pico de memória não fosse contaminada. Registraram-se tempo de treinamento, tempo de inferência, tamanho do modelo serializado e pico de memória (07_treino_final.py).

Etapa 9 — Avaliação. Leitura única do conjunto de teste, com predição sobre as 10,08 milhões de amostras. Cálculo das métricas globais e por classe, das matrizes de confusão, dos testes de Friedman e de Wilcoxon com correção de Holm, e do bootstrap por execução (08_avaliacao.py, 10_bootstrap_execucao.py).

5 RESULTADOS

5.1 Auditoria

A base confere com a descrição publicada. São 15,33 milhões de linhas, 52 variáveis idênticas entre treinamento e teste, 500 execuções por classe em todos os arquivos, amostras numeradas de 1 a 500 no treinamento e de 1 a 960 no teste. Não há valores ausentes, infinitos ou duplicatas de chave em nenhum dos quatro arquivos.

5.2 Análise exploratória

Sobre as 1,05 milhão de linhas de treinamento: duas componentes principais explicam 43,9% da variância e dez componentes explicam 74,5%, o que indica forte redundância entre as variáveis. Vinte e um pares de variáveis têm correlação absoluta acima de 0,9.

Medindo a distância média de cada falha ao regime normal, em desvios-padrão, as falhas mais próximas do normal são a 9 (0,007), a 14 (0,009), a 10 e a 15 (0,010) e a 19 (0,014). As mais distantes são a 6 (9,57), a 18 (4,47), a 2 (3,08) e a 1 (1,44). A análise antecipa, portanto, quais falhas serão difíceis antes de qualquer modelo ser treinado.

5.3 Efeito da unidade de divisão

Em dez repetições independentes, a divisão por amostra produziu resultado superior à divisão por execução em todas as combinações de repetição e modelo.

Tabela 2 — Efeito da unidade de divisão, dez repetições

Modelo	Por execução	Por amostra	Δ F1 macro [IC 95%]
Árvore (folha $\geq$ 50)	0,717	0,731	+0,013 [0,011; 0,016]
Árvore sem poda	0,705	0,732	+0,028 [0,025; 0,029]
Random Forest 50 (folha $\geq$ 50)	0,760	0,773	+0,013 [0,012; 0,015]
Random Forest 50 sem poda	0,778	0,807	+0,029 [0,028; 0,031]

Fonte: results/tables/tab_piloto_vazamento_resumo.csv.

O efeito dobra quando a poda é removida, o que é coerente com a explicação por memorização: quanto maior a capacidade do modelo de guardar exemplos, mais ele lucra com vizinhos temporais no teste. Todos os intervalos de confiança excluem zero, e o teste de Wilcoxon pareado retorna $p = 0,002$ nas quatro configurações.

5.4 Seleção de hiperparâmetros

Tabela 3 — Configurações escolhidas na validação

Modelo	Configuração	F1 macro na validação
Regressão logística	$C = 10$	0,464
Árvore de decisão	folha $\geq$ 20, sem limite de profundidade	0,743
Random Forest	200 árvores, folha $\geq$ 20, 30% dos atributos	0,788
Gradient boosting	taxa 0,05, 200 iterações, 31 folhas	0,745
XGBoost	500 árvores, profundidade 6, taxa 0,1	0,814
SVM (Nyström + SGD)	500 componentes, $\alpha = 10^{-5}$	0,631

Fonte: configs/hiperparametros_escolhidos.json.

Nas famílias baseadas em árvores, limitar a profundidade máxima mostrou-se muito mais prejudicial do que exigir um número mínimo de amostras por folha: para a árvore isolada, a diferença chegou a oito pontos de F1 macro. O gradient boosting revelou-se sensível à configuração, com quedas de até 0,05 ao passar a taxa para 0,1 ou o número de folhas para 63.

5.5 Desempenho no conjunto de teste

Tabela 4 — Métricas no teste, rótulo físico, média de cinco sementes

Modelo	Acurácia	Acurácia balanceada	F1 macro	MCC	F1 macro agregado [IC 95%]
XGBoost	0,764	0,815	0,817	0,750	0,817 [0,817; 0,818]
Random Forest	0,745	0,790	0,794	0,729	0,794 [0,793; 0,794]
Árvore de decisão	0,672	0,740	0,741	0,656	0,741 [0,740; 0,742]
Gradient boosting	0,666	0,699	0,706	0,645	0,701 [0,700; 0,702]
SVM (Nyström + SGD)	0,490	0,588	0,512	0,500	0,497 [0,496; 0,497]
Regressão logística	0,579	0,509	0,475	0,553	0,475 [0,474; 0,476]

Fonte: results/tables/tab03_metricas_globais.csv e tab10_bootstrap_execucao.csv.

O gradient boosting é o único modelo cuja posição depende da semente: o F1 macro variou entre 0,597 e 0,763, com desvio-padrão de 0,071. Os demais têm desvio-padrão inferior a 0,001, ou seja, são reprodutíveis até a terceira casa decimal.

O teste de Friedman rejeitou a igualdade entre os modelos, tanto para o F1 macro ($\chi^2 = 23,86$; $p < 0,001$) quanto para o MCC ($\chi^2 = 23,51$; $p < 0,001$). Nenhuma comparação pareada de Wilcoxon atingiu significância após a correção de Holm, o que é uma limitação estrutural: com cinco repetições, o menor p bilateral possível é 0,0625. O bootstrap por execução, com 2.000 reamostragens das 10.500 execuções de teste, produziu intervalos de 95% com largura inferior a 0,003, e as quinze diferenças entre pares de modelos excluíram o zero, a menor delas com 0,022.

5.6 Desempenho por classe

Tabela 5 — Classes mais difíceis, F1 médio entre os seis modelos

Classe	F1 médio
Falha 9	0,098
Falha 15	0,104
Falha 3	0,129
Normal (classe 0)	0,454
Falha 11	0,536

Fonte: results/tables/tab06b_dificuldade_por_classe.csv.

As falhas 3, 9 e 15 são as três que a análise exploratória apontara como as de menor afastamento do regime normal. Nem o melhor modelo as separa: o XGBoost obtém 0,222 na falha 3, 0,160 na falha 9 e 0,171 na falha 15, enquanto atinge 0,84 ou mais em todas as demais. A matriz de confusão mostra que essas três falhas e a condição normal formam um bloco fechado: 33% das amostras das falhas 9 e 15 e 30% das da falha 3 são atribuídas ao regime normal. Fora desse bloco, a matriz é quase diagonal.

5.7 Custo computacional

Tabela 6 — Custo no mesmo equipamento, média de cinco sementes

Modelo	Treinamento (s)	Inferência (μs por amostra)	Tamanho (MB)
Regressão logística	144,8	0,14	0,01
Árvore de decisão	83,9	0,16	1,8
SVM (Nyström + SGD)	54,6	2,18	1,1
Gradient boosting	291,5	43,28	6,6
XGBoost	525,3	31,44	13,0
Random Forest	1.266,2	9,68	286,2

Fonte: results/tables/tab05_custo_computacional.csv.

Os custos diferem por ordens de grandeza. O Random Forest é o mais caro para treinar e ocupa 286 MB, contra 1,8 MB da árvore isolada. O gradient boosting por histogramas, embora treine em menos de cinco minutos, tem a inferência mais lenta, porque avalia 21 árvores por iteração.

Na fronteira de Pareto entre F1 macro e tempo de inferência situam-se a regressão logística, a árvore de decisão, o Random Forest e o XGBoost; o gradient boosting e o SVM

aproximado são dominados. Ao longo dessa fronteira, o ganho de 7,7 pontos do XGBoost sobre a árvore custa uma inferência 194 vezes mais lenta e um modelo sete vezes maior.

6 DISCUSSÃO

Sobre QP1. O XGBoost obteve o melhor desempenho em todas as métricas, seguido do Random Forest e da árvore de decisão. A ordenação é estável: repetiu-se nas cinco sementes e sobreviveu à reamostragem das execuções de teste. Os conjuntos de árvores superaram o classificador linear em 32 a 34 pontos de F1 macro, o que confirma que a fronteira entre as classes deste problema não é linear.

Sobre QP2. As falhas 3, 9 e 15 permanecem confundidas com a operação normal por todos os métodos. A explicação não é do modelo, e sim dos dados: são justamente as falhas cujo afastamento médio do regime normal é da ordem de 0,01 desvio-padrão, medido antes de qualquer treinamento. Trabalhos independentes sobre a mesma base relatam o mesmo bloco de confusão, e parte da literatura simplesmente exclui essas três falhas da avaliação, o que infla o resultado publicado.

Sobre QP3. Dividir por amostra em vez de por execução infla o F1 macro entre 0,013 e 0,029, conforme a capacidade de memorização do modelo. O efeito é sistemático e estatisticamente sólido, mas de magnitude menor do que se costuma supor. A razão provável é que cada amostra, aqui, é o vetor instantâneo das 52 variáveis, sem janela temporal: amostras vizinhas são parecidas, não idênticas. Em trabalhos que constroem janelas deslizantes, amostras vizinhas compartilham a maior parte do conteúdo e o vazamento é muito maior.

Sobre QP4 e QP5. O custo separa os modelos com mais nitidez que o desempenho. A árvore de decisão entrega 91% do F1 macro do melhor modelo com uma inferência 194 vezes mais rápida e um modelo sete vezes menor. Como todos os modelos operam ordens de grandeza abaixo do intervalo de amostragem de três minutos do processo, a escolha entre os pontos da fronteira depende do equipamento disponível e do número de plantas a monitorar, não da capacidade de acompanhar o processo em tempo real.

Limitações. As execuções de treinamento foram mantidas fixas entre as sementes, de modo que a dispersão entre sementes mede a estabilidade do algoritmo, não a variabilidade decorrente de outras seleções de treinamento. As grades de hiperparâmetros foram reduzidas, e o gradient boosting e o SVM aproximado são os mais prejudicados por isso; seus resultados caracterizam as configurações adotadas, não as famílias. A árvore selecionada tem profundidade 82 e cerca de 18 mil folhas, de modo que seu baixo custo não implica legibilidade das regras. Todos os dados provêm de um único gerador, e a transferência a simulações independentes não foi avaliada.

7 DIFICULDADES ENCONTRADAS E AJUSTES

O registro completo, com 24 ocorrências, está em DESAFIOS-E-AJUSTES.md. As de maior efeito sobre o resultado foram:

SVM com núcleo exato inviável na escala. O custo entre quadrático e cúbico no número de amostras levou o computador a paginação intensa já com 30 mil amostras. Adotou-se a aproximação de Nyström do núcleo gaussiano seguida de SVM linear treinado por gradiente estocástico em lotes de 100 mil linhas. A implementação alternativa com LinearSVC sobre a matriz densa transformada exigiria mais de 8 GB e foi descartada.

Random Forest sem limite de crescimento não caberia na memória. A extrapolação a partir de árvores medidas indicava 57 GB para cem árvores. O crescimento passou a ser controlado pelo número mínimo de amostras por folha.

Falso positivo na verificação de vazamento. A primeira checagem acusou 3.150 pares de execução duplicados entre conjuntos. A causa era que os arquivos de treinamento e de teste numeram suas execuções de 1 a 500 de forma independente, e execuções com o mesmo número são simulações distintas. A chave de verificação passou a incluir o arquivo de origem.

Regressão logística sem convergir. O otimizador parava no teto de 300 iterações nas cinco sementes. A busca e o treinamento foram refeitos com teto de 5.000 iterações, e o modelo convergiu em 841 iterações. A configuração escolhida não mudou e o F1 macro subiu de 0,474 para 0,475, o que confirma que a conclusão sobre o modelo linear não dependia do erro, mas a afirmação só passou a ser defensável depois da correção.

Poder estatístico com cinco sementes. O Wilcoxon pareado bilateral com $n = 5$ tem p mínimo de 0,0625, de modo que nenhuma comparação poderia ser significativa após a correção de Holm. Acrescentou-se o bootstrap por execução, que responde à pergunta correta — quanto o resultado mudaria com outras execuções da planta — e produz intervalos que separam todos os pares de modelos.

Paralelismo não uniforme entre bibliotecas. Random Forest, XGBoost e o SVM aproximado rodaram com quatro fios de execução; a árvore de decisão é sequencial; o gradient boosting e as rotinas de álgebra linear usaram até oito fios, porque a variável de ambiente correspondente não foi fixada. Os tempos foram mantidos e a condição declarada, em vez de reexecutar todo o treinamento: a diferença entre os modelos é de ordens de grandeza e não se explica por esse fator.

8 CONCLUSÃO

O experimento produziu uma linha de base verificável para seis famílias clássicas de aprendizado de máquina no diagnóstico de falhas do Tennessee Eastman Process. Sobre 10,08 milhões de amostras de teste, provenientes de 10.500 execuções nunca vistas, o XGBoost obteve F1 macro de 0,817 e MCC de 0,750, seguido do Random Forest, com 0,794 e 0,729, e da árvore de decisão, com 0,741 e 0,656.

Três resultados merecem registro além do desempenho. Primeiro, as falhas 3, 9 e 15 permanecem indistinguíveis da operação normal por qualquer dos métodos, e a análise exploratória permitia prevê-lo. Segundo, a unidade de divisão dos dados altera o resultado medido entre 0,013 e 0,029 de F1 macro, e o rótulo atribuído às amostras anteriores à falha altera-o em até nove pontos: nenhuma comparação entre trabalhos é válida sem que essas

duas escolhas estejam declaradas. Terceiro, o custo computacional separa os modelos com mais nitidez que o desempenho, e a árvore de decisão isolada oferece 91% do F1 macro do melhor modelo a um custo de inferência próximo ao do classificador linear.

Todo o material está publicado em <https://github.com/ricardoarfr/MEI0028-modelagem-e-simulacao>, incluindo os scripts numerados, o manifesto de divisão, as previsões de cada modelo e semente sobre o teste completo e os metadados do ambiente, de modo que qualquer resultado aqui relatado pode ser reexecutado e conferido.

9 REFERÊNCIAS

BRODERSEN, K. H.; ONG, C. S.; STEPHAN, K. E.; BUHMANN, J. M. The balanced accuracy and its posterior distribution. In: INTERNATIONAL CONFERENCE ON PATTERN RECOGNITION, 20., 2010, Istanbul. **Proceedings** [...]. Piscataway: IEEE, 2010. p. 3121-3124. DOI: 10.1109/ICPR.2010.764.

CHICCO, D.; JURMAN, G. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. **BMC Genomics**, v. 21, n. 1, art. 6, 2020. DOI: 10.1186/s12864-019-6413-7.

DOWNS, J. J.; VOGEL, E. F. A plant-wide industrial process control problem. **Computers & Chemical Engineering**, v. 17, n. 3, p. 245-255, 1993. DOI: 10.1016/0098-1354(93)80018-I.

GORODKIN, J. Comparing two K-category assignments by a K-category correlation coefficient. **Computational Biology and Chemistry**, v. 28, n. 5-6, p. 367-374, 2004. DOI: 10.1016/j.compbiolchem.2004.09.006.

HAMMERLA, N. Y.; PLÖTZ, T. Let's (not) stick together: pairwise similarity biases cross-validation in activity recognition. In: ACM INTERNATIONAL JOINT CONFERENCE ON PERVASIVE AND UBIQUITOUS COMPUTING, 2015, Osaka. **Proceedings** [...]. New York: ACM, 2015. p. 1041-1051. DOI: 10.1145/2750858.2807551.

KAPOOR, S.; NARAYANAN, A. Leakage and the reproducibility crisis in machine-learning-based science. **Patterns**, v. 4, n. 9, art. 100804, 2023. DOI: 10.1016/j.patter.2023.100804.

RIETH, C. A.; AMSEL, B. D.; TRAN, R.; COOK, M. B. **Additional Tennessee Eastman Process simulation data for anomaly detection evaluation**. Harvard Dataverse, V1, 2017. Conjunto de dados. DOI: 10.7910/DVN/6C3JR1.

SAEB, S.; LONINI, L.; JAYARAMAN, A.; MOHR, D. C.; KORDING, K. P. The need to approximate the use-case in clinical machine learning. **GigaScience**, v. 6, n. 5, 2017. DOI: 10.1093/gigascience/gix019.